



PECUNIA

Gestión de Finanzas Personales

Curso 2025-2026

Ana Núñez Flores



Proyecto Intermodular Desarrollo de aplicaciones
Multiplataforma

www.enlaceGitHub.es

Agradecimientos

A mis profesores y tutores, por compartir su conocimiento y guiarme durante todo este camino. Vuestra ayuda ha sido fundamental para navegar entre las complejidades de Android Studio, logrando que conceptos que antes me resultaban ajenos se convirtieran en la base de esta aplicación.

A todas las personas, compañeros y familia, que han aportado su granito de arena para que Pecunia pasara de ser una idea a una realidad funcional. Este trabajo es mi resultado de un esfuerzo y de la ilusión por aprender.

Ana Núñez Flores

Mayo de 2026

ÍNDICE GENERAL

1. Introducción.....	5
1.1 El problema.....	5
1.2 Objetivo del proyecto.....	5
2. Análisis de Necesidades.....	6
2.1 Problema existente.....	6
2.1.1 Opacidad de los metadatos bancarios(Nombres Fiscales y nombres Comerciales):.....	6
2.1.2 Arquitectura de información orientada a la transacción:.....	7
2.1.3 El fenómeno del ‘Gasto Ciego’ y la falta de proactividad:.....	7
2.2 Soluciones actuales en el mercado.....	8
2.2.1 Herramientas ofimáticas y métodos tradicionales:.....	8
2.2.2 Aplicaciones de terceros y agregadores financieros.....	8
2.3 ¿Por qué Pecunia es útil?.....	8
2.3.1 Clasificación semántica e interfaz intuitiva:.....	9
2.3.2 Fomento de la consciencia financiera:.....	9
2.3.3 Sistema de alertas preventivas:.....	9
3. Plan de Empresa.....	10
3.1 Modelo de Negocio: Estrategia Freemium.....	10
3.2 Análisis del Público Objetivo(Target).....	11
3.3 Análisis de Mercado y Competencia.....	13
3.4 Estrategia de Marketing y Monetización Adicional.....	13
3.5 Análisis de Viabilidad y Previsión de Futuro.....	14
4. Tecnologías usadas.....	14
4.1 Entorno de Desarrollo(IDE) Android Studio.....	14
4.2 Lenguaje de programación: Java.....	14
4.3 Backend y Persistencia: Firebase.....	15
4.4 Control de versiones: Git y GitHub.....	15
4.5 Diseño Gráfico y Prototipado: Canva.....	15
4.6 Renderizado de Pantallas: HTML/WebView.....	16
5. Diseño del sistema.....	16
5.1 Diagrama E/R.....	16
5.2 Diagrama de clases(UML).....	17
5.3 Diagrama de actividades.....	18
5.4 Mockups del Sistema (Vistas Reales).....	19
5.4.1 Pantalla Login/Registro.....	20
5.4.2 Menú principal (Menú de acciones de operaciones).....	21
5.4.3 Formularios para operaciones.....	21
5.4.4 Resumen o balanza final.....	22
5.4.5 Interfaz gráfica Básica y Premium.....	23

6. Desarrollo técnico y Lógica de Negocio.....	25
6.1 Gestión de Usuarios y Persistencia en Firebase.....	25
6.1.1 Autenticación con Firebase Auth.....	25
6.2 Lógica de Control de Acceso y Funciones Premium.....	28
6.2.1 Validación de Roles en Tiempo Real.....	28
6.2.2 Adaptación Estética de la Interfaz(UI Customization).....	29
6.2.3 Aplicación transversal de la lógica de permisos.....	29
6.3 Integración de Servicios Externos y Geolocalización(WebService).....	30
6.3.1 Gestión de permisos y Ubicación GPS.....	30
6.3.2 Consumo de la API REST (OkHttp).....	31
6.3.3 Parseo de JSON y Actualización de la interfaz.....	32
6.4 Sistema de Alertas y Notificaciones Push(Firebase Cloud Messaging).....	34
6.4.1 Implementación del Servicio en Segundo Plano.....	34
6.4.2 Construcción de la Notificación y Canales de Alerta.....	35
7. Plan de Pruebas.....	36
7.1 Pruebas de Sistema y Gestión de Usuarios.....	36
7.1.1 Protocolo de Pruebas Ejecutadas.....	36
7.1.2 Análisis de Evidencias Visuales.....	38
7.2 Pruebas de lógica de Negocio y control Premium.....	41
7.2.1 Protocolo de pruebas realizadas.....	41
7.2.2 Análisis de Evidencias Visuales.....	42
7.3 Pruebas de integración con Servicios Externos.....	44
7.3.1 Protocolos de pruebas realizadas.....	44
7.3.2 Análisis de Evidencias Visuales.....	44
8. Conclusiones.....	45
9. Anexos.....	46
9.1 Glosario de términos.....	47
9.2 Bibliografía y Recursos.....	47

1. Introducción

En la última década, la sociedad ha experimentado una transformación digital sin precedentes que ha alterado radicalmente la forma en la que gestionamos nuestras finanzas. El uso del dinero en efectivo está siendo desplazado por métodos electrónicos, tarjetas de crédito y aplicaciones de pago instantáneo. Si bien esta digitalización aporta comodidad, también genera un fenómeno de ‘desconexión financiera’ directa al no visualizar físicamente el dinero, facilitando la aparición de lo que hoy se conoce como ‘gasto hormiga’.

La elección del nombre *Pecunia* para la aplicación y este proyecto no es casual, sino que responde a una voluntad de conectar la utilidad de la aplicación con las raíces históricas de la economía. El término proviene del latín que significa ‘dinero’ o riqueza. Es interesante destacar que esta palabra deriva originalmente de *pecus*, que significa ‘ganado’. En la Antigua Roma, antes de la generalización de la moneda metálica, la riqueza de una familia se medía por la cantidad de cabezas de ganado que poseía. Por tanto el término ‘Pecunia’ evoca el concepto primigenio de la gestión de activos y acumulación de valor.

1.1 El problema

El problema central identificado es la falta de control proactivo sobre el presupuesto mensual. La mayoría de los usuarios consulta su estado bancario de forma reactiva, es decir, una vez que el gasto ya se ha realizado y el saldo puede ser crítico.

1.2 Objetivo del proyecto

El presente proyecto tiene como objetivo principal el diseño, desarrollo e implementación de *Pecunia*, una aplicación móvil nativa para el sistema operativo Android. Pecunia busca empoderar al usuario en la gestión de su economía doméstica mediante una interfaz intuitiva y un sistema de notificaciones inteligentes.

Para alcanzar este objetivo, se establecen los siguientes objetivos específicos:

1. Desarrollo de una arquitectura robusta: Utilizar Android Studio y Java bajo una estructura de paquetes organizada y escalable.

2. Gestión de datos en la nube: Implementar Firebase Firestore como base de datos NoSQL para garantizar la sincronización de ingresos y gastos en tiempo real por Usuario.
3. Sistema de Seguridad y Roles: Integrar Firebase Authentication para permitir el acceso seguro y diferenciar las funcionalidades entre usuarios de tipo Básico y Premium.
4. Notificaciones proactivas: Desarrollar un servicio de alertas(Cloud Messaging y notificaciones locales) que informe al usuario de forma automática cuando su balance financiero sea inferior a un límite establecido.

2. Análisis de Necesidades

En este punto se va a realizar una valoración general sobre el sistema financiero del Usuario que vive actualizado con la tecnología.

2.1 Problema existente

Aunque la mayoría de las entidades bancarias ofrecen aplicaciones móviles para consultar el estado de la cuenta o el saldo, estas presentan barreras significativas para una gestión financiera eficiente:

2.1.1 Opacidad de los metadatos bancarios(Nombres Fiscales y nombres Comerciales):

Uno de los mayores obstáculos para el control de gastos es la discrepancia entre el nombre comercial de un establecimiento y su razón social o nombre fiscal. Cuando un usuario realiza una transacción, el sistema bancario registra los datos legales del Terminal de Punto de Venta(TPV). Esto provoca que en el extracto aparezcan denominaciones como 'Sociedad de Inversiones Hoteleras S.L' en lugar del nombre del restaurante o establecimiento visitado. Esta falta de normalización de gastos genera una carga cognitiva

innecesaria, obligando al usuario a realizar un ejercicio de memoria o investigación para identificar sus propios movimientos, lo que a menudo deriva en un abandono del seguimiento presupuestario.

2.1.2 Arquitectura de información orientada a la transacción:

Las aplicaciones bancarias tradicionales heredan estructuras de sistemas contables rígidos donde la información se presenta de forma cronológica y plana. Para un usuario que busca planificación, esta disposición es ineficiente, ya que mezcla sin distinción gastos fijos con gastos variables u 'hormiga' (gastos insignificantes que suman a final de mes una buena parte de los ingresos). La ausencia de una categorización semántica e

intuitiva impide que el usuario visualice, de un solo vistazo, el peso real de cada área de su vida sobre su saldo total del que dispone.

2.1.3 El fenómeno del 'Gasto Ciego' y la falta de proactividad:

El modelo actual de la banca es reactivo: el usuario solo conoce su situación si toma la iniciativa de entrar en la aplicación. No existe un sistema de alerta temprana que actúe antes de que el saldo llegue a niveles críticos. Además, debido al procesamiento de pagos (autorizaciones pendientes), el saldo que muestra el banco a veces no refleja las compras realizadas en las últimas 48h. Esta latencia informativa, sumada a la falta de notificaciones eficientes provoca que el usuario pierda el control real de su capacidad de gasto diaria hasta que es demasiado tarde para corregir el comportamiento del mes.

2.2 Soluciones actuales en el mercado

2.2.1 Herramientas ofimáticas y métodos tradicionales:

Muchos usuarios recurren a hojas de cálculo o incluso al registro en papel. Si bien estos métodos ofrecen una personalización total, presentan una barrera de entrada muy alta debido a la necesidad de conocimientos técnicos y a la falta de inmediatez. El registro manual diferido provoca que muchos gastos pequeños se olviden, invalidando el balance final del mes y generando frustración en el usuario.

2.2.2 Aplicaciones de terceros y agregadores financieros.

Existen aplicaciones comerciales que intentan centralizar la información bancaria. Sin embargo, suelen pecar de un exceso de complejidad al incluir funciones de inversión, bolsa o criptomonedas, lo que abruma al usuario medio que solo busca un control doméstico. Además, muchas de estas herramientas son percibidas como invasivas al

solicitar credenciales bancarias directas, lo que genera reticencia por motivos de seguridad y privacidad de datos.

2.3 ¿Por qué Pecunia es útil?

Pecunia surge no como una herramienta contable más, sino como un asistente de salud financiera diseñado para cerrar la brecha entre el gasto real y la percepción del usuario. Su utilidad reside en tres pilares fundamentales:

2.3.1 Clasificación semántica e interfaz intuitiva:

A diferencia de los extractos bancarios crudos, Pecunia ofrece una capa de interpretación visual. Al permitir clasificar los gastos en categorías humanas (Alimentación, Salud, Ocio, Transporte...) mediante una interfaz limpia y minimalista, el usuario puede identificar patrones de consumo de forma instantánea. Esta organización ayuda a detectar fugas de capital en áreas no esenciales que, de otro modo, quedarían ocultas bajo nombres fiscales ininteligibles.

2.3.2 Fomento de la consciencia financiera:

Pecunia apuesta por el registro activo. Al obligar a usar al usuario a introducir sus movimientos, se rompe la inercia del pago automático con tarjeta o dispositivos móviles. Este pequeño gesto de registro manual actúa como un mecanismo psicológico que devuelve el control al individuo, obligándole a reconocer el gasto en el momento en que se produce y fomentando así una conducta de ahorro más responsable.

2.3.3 Sistema de alertas preventivas:

La característica más diferenciadora y útil de la aplicación es su proactividad. Mientras que en otras aplicaciones son consultivas (hay algunas que ya han implementado esta novedad), Pecunia es comunicativa. Gracias a la implementación de servicios de mensajería y lógica interna, la aplicación avisa al usuario mediante una notificación cuando el balance es inferior a 50€. Este 'umbral de seguridad' sirve como un fenómeno preventivo, permitiendo al usuario reajustar sus prioridades antes de que finalice el mes, evitando así el riesgo de entrar en descubierto bancario.

3. Plan de Empresa

3.1 Modelo de Negocio: Estrategia Freemium

El modelo de negocio de Pecunia se basa en la estrategia Freemium, diseñada para maximizar la adquisición de usuarios sin renunciar a la rentabilidad.

- Pecunia Basic(Adquisición): Se ofrece de forma gratuita con el objetivo de eliminar barreras de entrada. Incluye funciones críticas: registro de transacciones, cálculo de balance y el sistema de alertas de seguridad. La gratuidad permite generar una base de datos de usuario activa que nutre el ecosistema de la aplicación.
- Pecunia Premium(Monetización): Mediante una suscripción mensual o anual (ej. 1,99€/mes), el usuario accede a herramientas de análisis avanzado. Esto incluye el desglose por categorías(Alimentación, Ocio, Transporte...), generación de informes mensuales y la eliminación de posible publicidad. Este nivel está orientado a usuarios que buscan una optimización real de su ahorro.

Funcionalidad	Pecunia Básico	Pecunia Premium
Registro de ingresos y gastos	✓	✓
Cálculo de balance en tiempo real	✓	✓
Alertas de saldo bajo (<50€)	✓	✓
Clasificación por categorías	✗	✓
Historial de años anteriores	✗	✓
Exportación de informes (PDF/Excel)	✗	✓
Eliminación de publicidad	✗	✓
Soporte prioritario	✗	✓

Figura 1. Tabla visual roles de usuario

3.2 Análisis del Público Objetivo(Target)

Pecunia no pretende ser una app para contadores profesionales, sino para personas que sufren el día a día de la gestión financiera. He identificado tres perfiles clave:

1. Estudiantes y jóvenes: Personas con presupuestos muy ajustados donde un error de cálculo de 20€ puede suponer no llegar a fin de mes. Para ellos, la notificación de 'Saldo bajo' es la función perfecta.
2. Freelancers y Autónomos: Profesionales con ingresos irregulares que necesitan saber en todo momento cuánto dinero 'real' tienen disponible para gastos personales tras apartar sus obligaciones fiscales.
3. Familias en búsqueda de ahorro: Hogares que necesitan identificar el 'Gasto hormiga' mediante una categorización detallada.

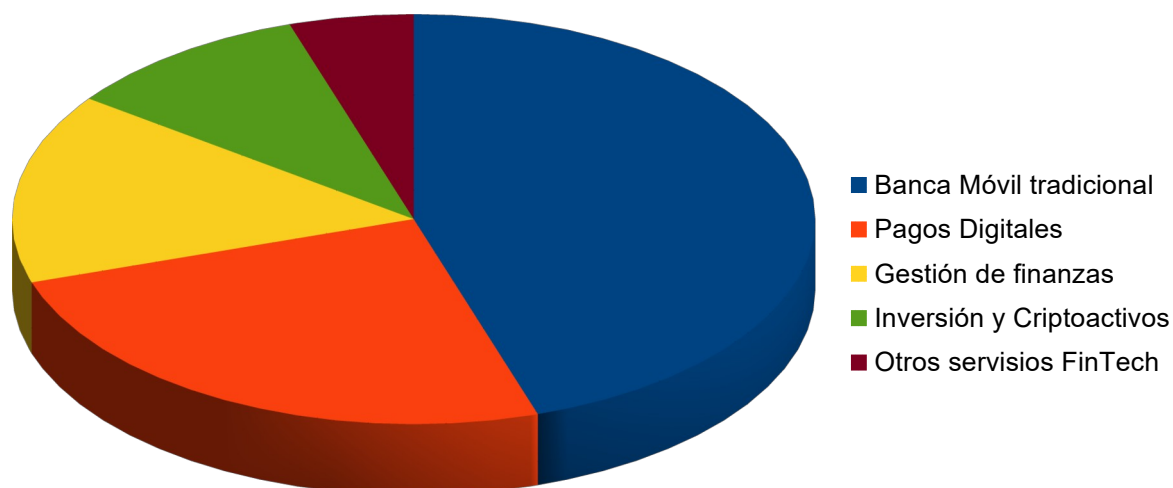


Figura 2. Cuota de mercado de apps financieras.

Como se observa en el gráfico anterior, aunque la Banca Móvil domina el mercado, existe un sólido 15% de usuarios que demandan herramientas específicas de gestión de

finanzas personales (PFM). Pecunia se enfoca en este segmento, captando a aquellos usuarios que, a pesar de tener la app de su banco, necesitan una capa extra de categorización y alertas proactivas que las entidades tradicionales no ofrecen. Este nicho representa una oportunidad de crecimiento anual del 8% debido a la creciente preocupación por el ahorro.

3.3 Análisis de Mercado y Competencia

Para validar la viabilidad, he analizado el sector FinTech actual. Mientras que las apps de los grandes bancos(Santander, BBVA, CaixaBank) son potentes, su interfaz es a menudo abrumadora y poco ‘humana’.



Figura 3. Análisis DAFO de la aplicación

3.4 Estrategia de Marketing y Monetización Adicional

Además de las suscripciones, se plantean vías de ingresos laterales:

- **Publicidad Nativa:** En la versión básica se mostrarán anuncios relacionados exclusivamente con el sector financiero(seguro, cuentas de ahorro, depósitos), integrados de forma que no interrumpan la experiencia del usuario.
- **Marketing de Afiliación:** Pecunia puede sugerir servicios. Por ejemplo, si la app detecta un gasto recurrente muy alto en la categoría 'Hogar/Electricidad', podría mostrar un enlace a un comparador de tarifas eléctricas, llevándose una comisión por cada alta.

3.5 Análisis de Viabilidad y Previsión de Futuro

La viabilidad técnica está garantizada por el uso de tecnologías escalables como Firebase. Desde el punto de vista comercial, Pecunia es escalable mediante la implementación de Open Banking. Esta tecnología permitirá en el futuro que la app se conecte directamente a la cuenta del usuario, categorizando los gastos de forma automática mediante algoritmos de aprendizaje automático (Machine Learning). Esta evolución permitiría pasar de una herramienta de registro manual a un asistente financiero autónomo, aumentando significativamente el valor de la suscripción Premium.

4. Tecnologías usadas

En este apartado se describen las herramientas y lenguajes que han permitido el desarrollo de Pecunia, justificando la elección de cada una de ellas por su robustez y escalabilidad.

4.1 Entorno de Desarrollo(IDE) Android Studio

Para el desarrollo de la aplicación se ha usado Android Studio, el entorno de desarrollo integrado oficial de Google. Su elección se basa en la integración nativa con el SDK de Android y herramientas avanzadas de depuración(Logcat) y perfilado de rendimiento.

Para las pruebas se ha usado el mismo emulador que contiene la herramienta junto con móvil Android para la visión real de las pantallas en dispositivos.

4.2 Lenguaje de programación: Java

Aunque Kotlin es el lenguaje de referencia, se ha optado por Java debido a su madurez y a la sólida comprensión de su programación orientada a objetos(POO). Se ha utilizado características de Java 8 para la gestión de colecciones y flujos de datos. También ha sido el lenguaje en el que nos hemos especializado durante el curso.

4.3 Backend y Persistencia: Firebase¹

Como infraestructura de servidor se ha utilizado Google Firebase, lo que permite una arquitectura Serverless (sin servidor rápido). Dentro de este ecosistema se han empleado:

- Firebase Authentication: Para la gestión segura de cuentas de usuario.
- Cloud Firestore: Base de datos NoSQL en tiempo real para almacenar los ingresos y gastos de forma sincronizada.
- Firebase Cloud Messaging: Para el envío de notificaciones de umbral crítico.

4.4 Control de versiones: Git y GitHub

Se ha empleado Git como sistema de control de versiones, utilizando la plataforma GitHub para el alojamiento del código fuente. Esto ha permitido mantener un historial de cambios detallado y asegurar la integridad del código durante todo el ciclo de desarrollo.

¹ Se ha optado por Cloud Firestore frente a SQLite (la base de datos local nativa de Android) debido a la necesidad de persistencia en la nube. Mientras que SQLite almacena los datos únicamente en el dispositivo físico, Firebase permite que el usuario acceda a su información desde cualquier dispositivo tras autenticarse, garantizando la seguridad de los datos ante la pérdida o cambio del dispositivo. También permite al desarrollador control total sobre los usuarios de Pecunia.

4.5 Diseño Gráfico y Prototipado: Canva

Para la creación de la identidad visual de la aplicación, incluyendo el logotipo (Pecunia) y los elementos gráficos de la interfaz (iconos, banners, recursos visuales), se ha utilizado en su mayoría Canva.

Esta herramienta ha permitido mantener una coherencia estética en toda la documentación y en la propia aplicación, facilitando el diseño de una interfaz de usuario (UI) moderna y amigable.

4.6 Renderizado de Pantallas: HTML/WebView

Para el contenido y diseño visual de las screens se ha integrado contenido HTML. Esta estructura técnica permite mostrar la información ordenada, ajustar los espacios y el diseño de la misma pantalla. Esto combina la potencia del desarrollo nativo en Java con la flexibilidad del diseño web para contenidos específicos.

5. Diseño del sistema

5.1 Diagrama E/R

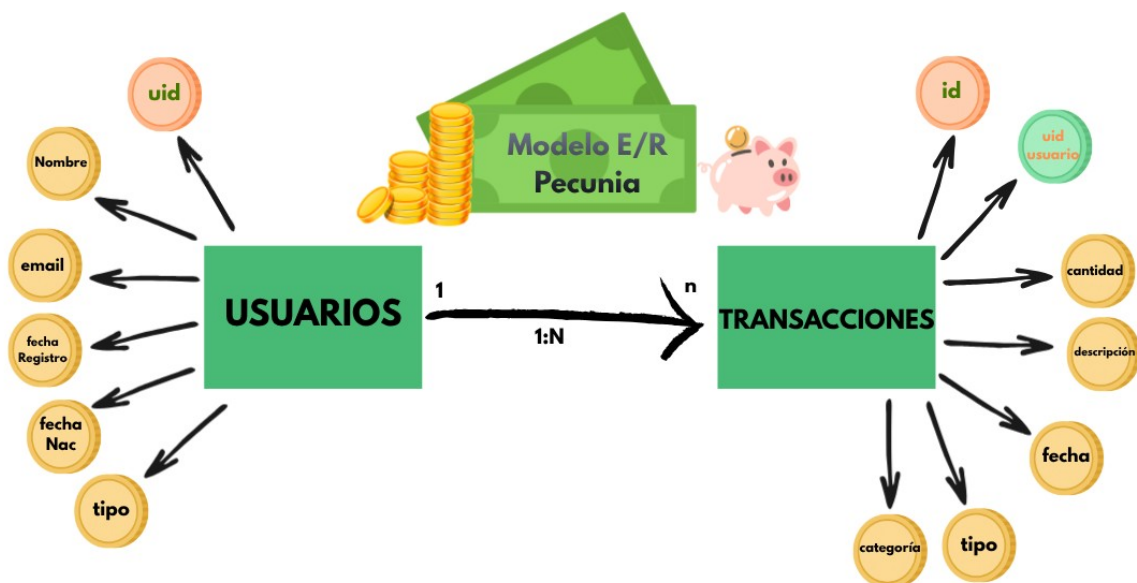


Figura 4. Modelo Entidad Relación de Pecunia

En el diagrama superior se detalla la arquitectura de datos de Pecunia. Se observa una relación 1:N (uno a muchos) entre la entidad de Usuarios y Transacciones, lo que permite que cada perfil almacene un histórico ilimitado de movimientos financieros. Cabe destacar el uso de claves externas(uid usuario) para garantizar la integridad referencial en Firebase y el atributo tipo en transacciones, que actúa como discriminador para procesar tanto ingresos como gastos bajo una misma estructura lógica. Se señalan las PK con texto verde y la FK moneda verde con texto rojo.

5.2 Diagrama de clases(UML)

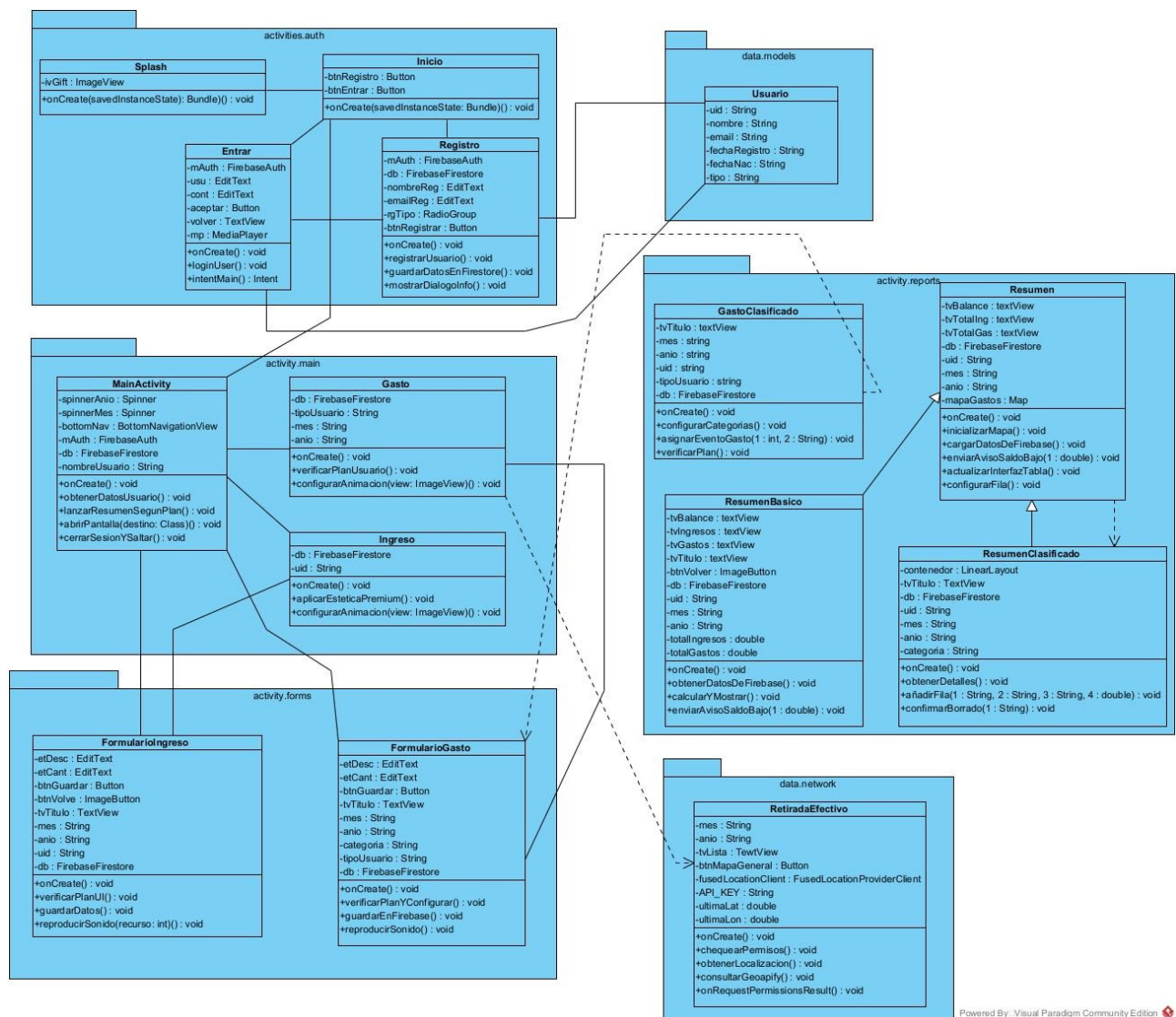


Figura 5. Diagrama de clases del sistema Pecunia

Este diagrama representa la arquitectura lógica del sistema, estructurada mediante un enfoque orientado a objetos para garantizar la escalabilidad y el mantenimiento del código. El sistema se divide en tres capas principales:

- Capa de Presentación(activities): Organizada en subpaquetes(auth, main, forms, reports), esta capa gestiona la interfaz de usuario y la navegación entre pantallas. Se utiliza la herencia para optimizar la lógica común de resúmenes financieros y la composición para delegar responsabilidades en componentes específicos.
- Capa de Negocio y Datos(data.models): Define las entidades persistentes, siendo la clase Usuario el pilar fundamental que almacena el estado y los metadatos de cada cuenta.
- Capa de Red y Servicios(data.network): Encapsula la lógica de comunicación externa como el consumo de APIs geoespaciales(Geoapify) para la localización de cajeros, desacoplado la funcionalidad de red en la interfaz gráfica.

5.3 Diagrama de actividades

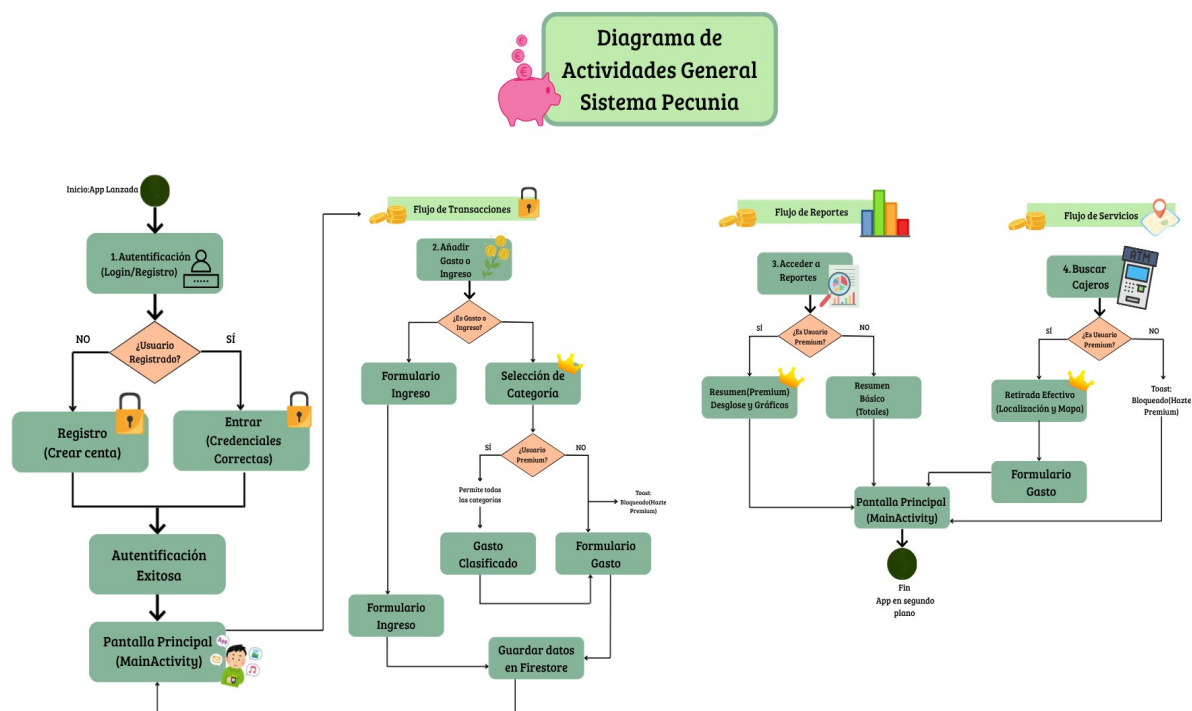


Figura 6. Diagrama de actividades Pecunia

Este diagrama describe el flujo dinámico de ejecución del sistema, detallando el camino que sigue el usuario a través de los diferentes procesos funcionales de la aplicación:

- **Gestión de Acceso(Autenticación):** El flujo inicial garantiza la seguridad financiera mediante la identificación obligatoria en cada inicio. Dependiendo del estado del usuario (registrado o nuevo), el sistema redirige el proceso hacia la validación de credenciales (Login) o a la creación de un nuevo perfil (Registro), consolidando el acceso en la pantalla principal (MainActivity).
- **Flujo de Transacciones:** Representa el núcleo operativo de Pecunia. Este proceso gestiona la entrada de gastos e ingresos, aplicando reglas de negocio críticas para la monetización. Se integra un mecanismo de validación que consulta el rol de usuario en Firestore, bloqueando o permitiendo el acceso a categorías específicas basándose en el plan contratado.
- **Flujo de Reportes y Servicios:** Describe la capacidad analítica del sistema, ramificándose entre una vista simplificada (Resumen Básico) y una avanzada (Resumen Premium), además de la integración con servicios externos de la localización (API de Geoapify) para la búsqueda de cajeros cercanos.

Este diagrama no solo ilustra la navegación entre actividades, sino que formaliza la lógica condicional que dicta el comportamiento de la aplicación en tiempo de ejecución, asegurando que las restricciones de acceso y las funcionalidades de valor añadido se apliquen correctamente según el contexto del usuario.

5.4 Mockups del Sistema (Vistas Reales)

En esta sección se presentan capturas de pantalla reales de la aplicación Pecunia en funcionamiento. Estas vistas documentan la interfaz de usuario final y demuestran la implementación de la lógica de negocio descrita en los diagramas anteriores.

5.4.1 Pantalla Login/Registro

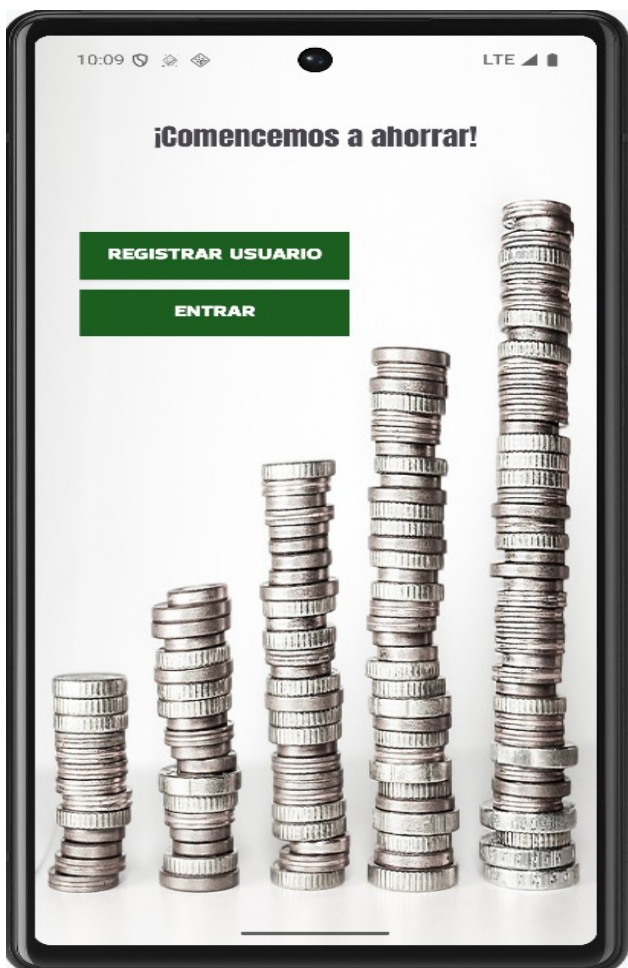
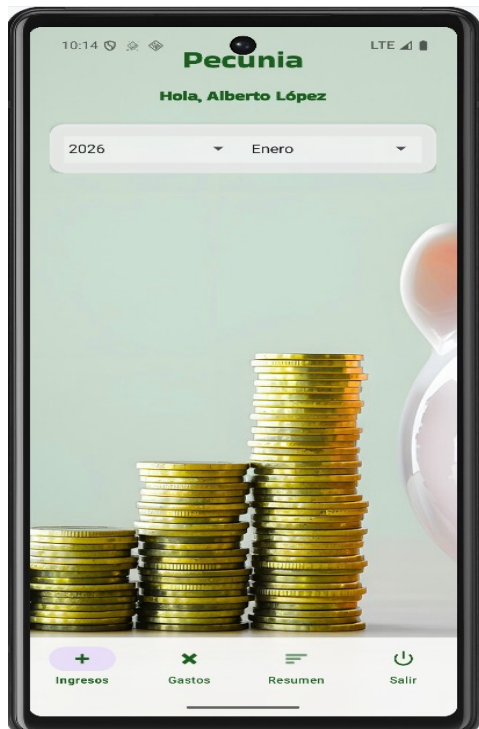


Figura 7. Inicio de la aplicación

Esta es la pantalla principal donde el Usuario decide si registrar una cuenta nueva (en caso de no tenerla o querer otra para otras gestiones) o entrar con sus credenciales a la app.

5.4.2 Menú principal (Menú de acciones de operaciones)



Una vez que se inicia después del Login, aparece el menú principal de esta aplicación donde el Usuario elige qué tipo de operación desea hacer seleccionando el mes y año en el que quiere guardar los datos. Esta selección, es la que durante todas las actividades va a captar desde aquí para que todos los datos que se guarden en las colecciones de Firestore estén en su correspondiente fecha elegida.

Figura 8. Menú principal

5.4.3 Formularios para operaciones

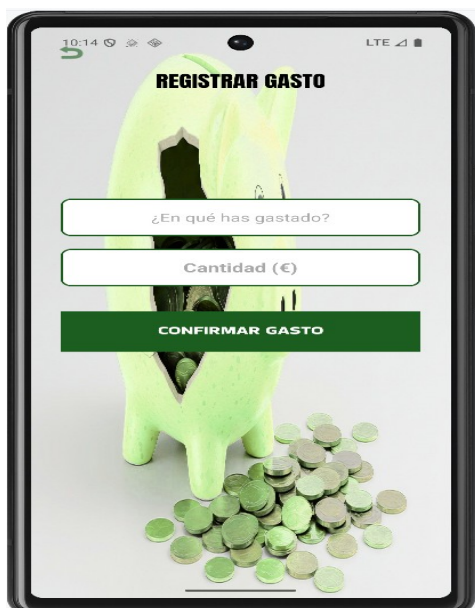


Figura 9. Formulario Gastos

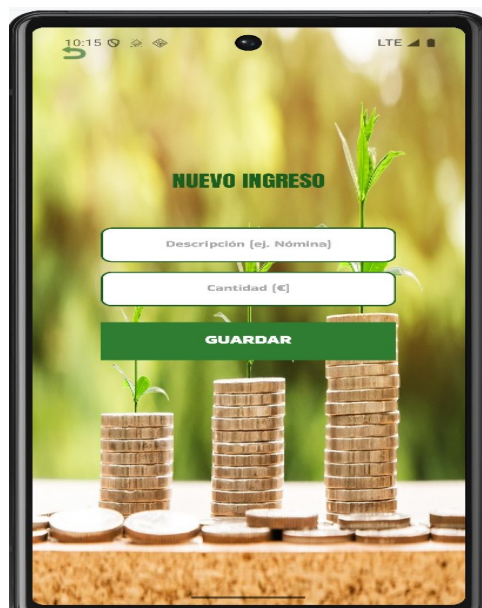


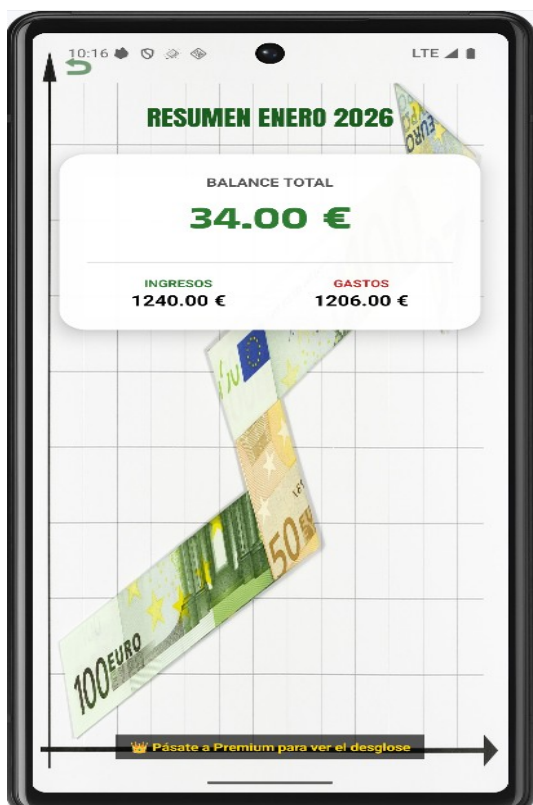
Figura 10. Formulario Ingresos

Independientemente de qué tipo de Usuario sea (Básico o Premium) después de pasar por los procesos Premium se llega siempre a estos dos únicos formularios de operaciones, uno para los ingresos y otro para los gastos.

En ambos se puede añadir una descripción o nombre para localizar que gasto es, por ejemplo, una limpieza dental. El sistema capta la fecha de manera automática en la que se realiza el movimiento introduciendo por último la cantidad de este.

Una vez se le da a guardar, se guardan los datos directamente en la consola de nuestro FirebaseFirestore en diferentes colecciones ordenado por fecha. Por último, se vuelve a la pantalla principal donde encontramos de nuevo el menú visto anteriormente.

5.4.4 Resumen o balanza final



Este es el caso de un resumen normal (Usuario básico) donde se calcula en un mismo mes la diferencia entre la suma total de los ingresos y la suma total de los gastos.



Pecunia incluye este tipo de mensajes que van directamente al sistema del dispositivo en el que se encuentre, en el que se avisa que el saldo es inferior a una cantidad (50€).

Figura 11. Balance mensual

5.4.5 Interfaz gráfica Básica y Premium

Para los diferentes roles de Usuario he identificado ciertos elementos diferenciales entre pantallas según el tipo. Este dato lo identifica el sistema consultando el formulario del Usuario al registrarse donde elige si quiere ser Premium o Básico.



Figura 12. Interfaz Premium Inicio

Como vemos en la imagen, los Usuarios Premium se diferencian por el color dorado y el icono de la corona en todas las actividades y pantallas.



Figura 13. Interfaz Premium gastos



Figura 14. Interfaz Premium API

Otras diferencias claves entre Usuarios son las actividades bloqueadas. Para un Usuario Básico es denegada la entrada a este tipo de pantallas en la aplicación en las que se le permite clasificar sus gastos mediante un menú de tipos de gastos. Otra función es el uso de la ubicación para localizar cajeros y poder registrar otro tipo de gasto en el formulario.

Para rematar la diferencia de roles, se ha realizado dos tipos de balance, uno en el que el Usuario puede ver los gastos clasificados situados en una tabla visual y otro en el que solo van a aparecer los totales como hemos visto anteriormente. Otro detalle de esta pantalla es que si el Usuario Premium pulsa alguna de las categorías de la tabla del Resumen, se redirige a una pantalla diferente en la que dentro de esa categoría aparecen todos los gastos con su descripción y fecha.



Figura 15. Balance Premium

6. Desarrollo técnico y Lógica de Negocio

6.1 Gestión de Usuarios y Persistencia en Firebase

En este apartado se explica cómo la aplicación gestiona el acceso de los usuarios y cómo se estructuran sus datos en la nube.

6.1.1 Autenticación con Firebase Auth

El sistema utiliza Firebase Auth para gestionar el acceso. Esto garantiza que las contraseñas no se almacenen en texto plano y que cada usuario tenga un identificador único inalterable.

Tanto para el registro como para el login con credenciales ya creadas en cada caso instanciamos para la base de datos y la autenticación en Firebase.

3 usages

```
private FirebaseAuth mAuth;
```

2 usages

```
private FirebaseFirestore db;
```

```
private FirebaseAuth mAuth;
```

Proceso de Registro de nuevos usuarios:

El registro es el primer punto de contacto donde se crea la identidad digital del usuario. Este proceso es híbrido: primero se crea la credencial en el motor de autenticación y, posteriormente, se inicializa su perfil en la base de datos.

En este fragmento se observa la captura de información mediante los componentes EditText. Un punto clave es el uso de RadioGroup para determinar el tipoUsuario. Mediante un operador ternario, se asigna el valor Premium o Básico, el cual será determinante para la lógica de negocio posterior. Además, se aplican validaciones de seguridad mediante estructuras if, comprobando la longitud de la contraseña y la

integridad de los datos (comparando que ambas contraseñas coincidan), evitando peticiones innecesarias al servidor si los datos son erróneos.

```
private void registrarUsuario() {
    final String nombre = nombreReg.getText().toString().trim();
    final String fechaNac = fechaReg.getText().toString().trim();
    final String email = emailReg.getText().toString().trim();
    String pass = passReg.getText().toString().trim();
    String conf = passConf.getText().toString().trim();

    // Capturar el tipo de usuario del RadioGroup
    int selectedId = rgTipo.getCheckedRadioButtonId();
    final String tipoUsuario = (selectedId == R.id.rbPremium) ? "Premium" : "Básico";

    // 1. Validar campos vacíos (ahora incluimos nombre y fecha)
    if (nombre.isEmpty() || fechaNac.isEmpty() || email.isEmpty() || pass.isEmpty() || conf.isEmpty()) {
        Toast.makeText(context, this, text: "Por favor, completa todos los campos", Toast.LENGTH_SHORT).show();
        return;
    }

    // 2. Validar longitud mínima
    if (pass.length() < 6) {
        Toast.makeText(context, this, text: "La contraseña debe tener al menos 6 caracteres", Toast.LENGTH_SHORT).show();
        return;
    }

    // 3. Validar que coincidan
    if (!pass.equals(conf)) {
        Toast.makeText(context, this, text: "Las contraseñas no coinciden", Toast.LENGTH_SHORT).show();
        return;
    }
}
```

Se emplea el método `createUserWithEmailAndPassword(email, pass)`. Lo más relevante aquí es el uso de un `OnCompleteListener`. Al ser una operación de red asíncrona, el sistema queda a la escucha del resultado sin bloquear la interfaz. Si `task.isSuccessful()` devuelve true, el sistema dispara automáticamente el método `guardarDatosEnFirestore()`, encadenando la creación de la cuenta con la persistencia de sus datos personales.

```
mAuth.createUserWithEmailAndPassword(email, pass)
    .addOnCompleteListener( activity: this, new OnCompleteListener<AuthResult>() {
        @Override
        public void onComplete(@NonNull Task<AuthResult> task) {
            if (task.isSuccessful()) {
                // Si se crea en Auth, procedemos a guardar en Firestore
                guardarDatosEnFirestore(nombre, email, fechaNac, tipoUsuario);
            } else {
                Toast.makeText(context: Registro.this, text: "Error al registrar: " + task.getException().getMessage(),
                    Toast.LENGTH_LONG).show();
            }
        }
    });
}
```

Por último guardamos los datos en FirebaseFirestore. Aquí se utiliza el método `.collection("Usuarios").document(uid).set(nuevoUsuario)`. Es fundamental destacar el uso del `document(uid)`: el identificador único generado por Firebase Auth se utiliza como clave

primaria en Firestore. Esto garantiza una vinculación unívoca entre la cuenta y sus datos financieros. Al finalizar con éxito, se emplea Intent para navegar hacia el MainActivity y se invoca el método *finish()* para destruir la actividad de registro y evitar que el usuario pueda volver atrás una vez iniciada la sesión.

```
db.collection( collectionPath: "Usuarios").document(uid).set(nuevoUsuario)
    .addOnCompleteListener(new OnCompleteListener<Void>() {
        @Override
        public void onComplete(@NonNull Task<Void> task) {
            if (task.isSuccessful()) {
                Toast.makeText( context: Registro.this, text: ";Bienvenido a Pecunia!", Toast.LENGTH_SHORT).show();
                startActivity(new Intent( packageContext: Registro.this, MainActivity.class));
                finish();
            } else {
                Toast.makeText( context: Registro.this, text: "Error: " + task.getException().getMessage(), Toast.LENGTH_SHORT).show();
            }
        }
    });
}
```

Proceso de Autenticación(Login)

El acceso a la aplicación se gestiona de forma centralizada para garantizar que solo los usuarios autorizados puedan consultar la información financiera.

```
mAuth.signInWithEmailAndPassword(email, password)
    .addOnCompleteListener( activity: this, Task<AuthResult> task -> {
        if (task.isSuccessful()) {
            // --- GUARDAR EL TOKEN EN FIRESTORE ---
            String userId = mAuth.getCurrentUser().getUid();
            FirebaseMessaging.getInstance().getToken()
                .addOnCompleteListener( Task<String> taskToken -> {
                    if (taskToken.isSuccessful()) {
                        String token = taskToken.getResult();
                        // Cambia "Usuarios" por el nombre exacto de la colección en Firestore
                        FirebaseFirestore.getInstance().collection( collectionPath: "Usuarios") CollectionReference
                            .document(userId) DocumentReference
                            .update( field: "fcmToken", token) Task<Void>
                            .addOnSuccessListener( Void aVoid -> Log.d( tag: "FCM", msg: "Token guardado"))
                            .addOnFailureListener( Exception e -> Log.e( tag: "FCM", msg: "Error al guardar token", e));
                    }
                });
        }
    });
```

El login utiliza el servicio de autenticación para verificar las credenciales introducidas. Al invocar *mAuth.signInWithEmailAndPassword(email, pass)*, Firebase busca en su base de datos de usuarios registrados. Si la validación es positiva, la aplicación recupera el estado de sesión del usuario. Una ventaja técnica implementada es que, tras el login exitoso, el sistema ya tiene acceso al UID, lo que permite cargar de forma inmediata los gastos e ingresos correspondientes a ese usuario específico en las pantallas principales.

6.2 Lógica de Control de Acceso y Funciones Premium

Pecunia implementa un modelo de negocio freemium basado en roles de usuario. La aplicación no solo almacena el tipo de cuenta, sino que consulta activamente en cada interacción crítica para decidir qué funciones habilitar.

6.2.1 Validación de Roles en Tiempo Real

A diferencia de otras aplicaciones que solo bloquean el acceso al inicio, Pecunia realiza una comprobación dinámica. Cuando un usuario intenta clasificar un gasto, el sistema recupera el atributo *tipo* desde el documento de usuario en Firestore.

```
private void asignarEventoGasto(int idCard, String nombreCategoria) {
    CardView card = findViewById(idCard);
    card.setOnClickListener( View v -> {

        // Si el usuario es básico y elige algo que no sea Alimentación o Salud...
        if ("Básico".equals(tipoUsuario) &&
            (!nombreCategoria.equals("Alimentación") && !nombreCategoria.equals("Salud"))) {

            Toast.makeText( context: this, text: "👑 Categoría bloqueada. ¡Hazte Premium!", Toast.LENGTH_SHORT).show();
        } else {
            // Si tiene permiso, vamos al formulario final
            Intent i = new Intent( packageContext: GastoClasificado.this, FormularioGasto.class);
            i.putExtra( name: "MES_SELECCIONADO", mes);
            i.putExtra( name: "ANIO_SELECCIONADO", anio);
            i.putExtra( name: "CATEGORIA", nombreCategoria); // Pasamos la categoría seleccionada
            startActivity(i);
        }
    });
}
```

6.2.2 Adaptación Estética de la Interfaz(UI Customization)

Además de bloquear funciones, el código modifica la apariencia de la aplicación para incentivar la suscripción y dar feedback visual del estatus del usuario.

```
private void verificarPlan() {
    if (uid == null) return;

    db.collection( collectionPath: "Usuarios").document(uid).get().addOnSuccessListener( DocumentSnapshot documentSnapshot -> {
        if (documentSnapshot.exists()) {
            tipoUsuario = documentSnapshot.getString( field: "tipo");

            // Si es Premium, cambiamos el título a Dorado con corona
            if ("Premium".equals(tipoUsuario)) {
                tvTitulo.setText("CATEGORÍAS PREMIUM 👑");
                tvTitulo.setTextColor(Color.parseColor( colorString: "#B8860B")); // Dorado
            }
        }
    });
}
```

6.2.3 Aplicación transversal de la lógica de permisos

Para mantener la coherencia de modelo de negocio, esta validación del campo *tipoUsuario* no se limita a la clasificación de gastos, sino que se extiende de forma transversal a otros módulos de la aplicación:

- Buscador de Cajeros(RetiradaEfectivo): Aunque la clase usa servicios de localización y APIs externas, el acceso al formulario de registro de dicha retirada está condicionado. Se utiliza la misma lógica de validación para asegurar que solo los usuarios Premium puedan consultarlo.
- Módulo de Informes Avanzados(ResumenClasificado): Mientras el ResumenBasico muestra totales generales, el acceso a la clase ResumenClasificado(que desglosa gastos por categorías) está restringido.

En lugar de duplicar el código, la aplicación utiliza el objeto Usuario(definido en el paquete `data.models`) como una entidad persistente durante la sesión. Esto permite que cualquier Activity pueda invocar el atributo `.getTipo()` y aplicar reglas de visibilidad o de interacción bajo el mismo criterio de negocio, facilitando el mantenimiento del software y la escalabilidad del sistema *freemium*.

6.3 Integración de Servicios Externos y Geolocalización(WebService)

El módulo de “Retirada de Efectivo” representa la capacidad de la aplicación para interactuar con servicios externos. El objetivo es que el usuario encuentre cajeros cercanos(ATMs) de forma automática mediante la integración de la API de Geoapify.

6.3.1 Gestión de permisos y Ubicación GPS

Antes de realizar cualquier consulta, la aplicación debe interactuar con el hardware del dispositivo.

```
private void comprobarPermisos() {  
    if (ActivityCompat.checkSelfPermission( context: this, Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {  
        ActivityCompat.requestPermissions( activity: this, new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, requestCode: 100);  
    } else {  
        obtenerLocalizacion();  
    }  
}
```

```
private void obtenerLocalizacion() {  
    if (ActivityCompat.checkSelfPermission( context: this, Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {  
        return;  
    }  
  
    fusedLocationClient.getLastLocation().addOnSuccessListener( activity: this, Location location -> {  
        if (location != null) {  
            ultimaLat = location.getLatitude();  
            ultimaLon = location.getLongitude();  
            consultarGeoapify(ultimaLat, ultimaLon);  
        } else {  
            tvLista.setText("No se puede obtener la ubicación. Verifica que el GPS esté activo.");  
        }  
    });  
}
```

Para garantizar la privacidad, se implementa el flujo de permisos en tiempo de ejecución (*Manifest.permission.ACCESS_FINE_LOCATION*). Se utiliza la librería de Google Play

Services (*FusedLocationProviderClient*) porque es la forma más eficiente de obtener la última ubicación conocida del dispositivo sin agotar la batería. El uso del método *addOnSuccessListener* asegura que la consulta de la API solo se dispare cuando el dispositivo devuelva coordenadas de latitud y longitud válidas.

6.3.2 Consumo de la API REST (OkHttp)

Una vez obtenidas las coordenadas, la aplicación se comporta como un cliente web.

Se ha utilizado la librería OkHttp por su robustez y manejo de asincronía. Un aspecto clave aquí es el uso del método *.enqueue()*. Al ser una petición a un servidor externo, no se puede realizar en el hilo principal(Main Thread) ya que congelaría la aplicación.²

- Construcción de la URL: Se observa una petición GET donde se pasan parámetros como el radio de búsqueda (2000 metros) y la categoría específica *service.financial.atm*.
- Seguridad: Se usa una API_KEY para autenticar las peticiones ante el servidor de Geoapify.

² Es fundamental comprender que Android Prohíbe las operaciones de red en el hilo de interfaz de usuario. Si se intenta realizar esta petición de forma síncrona, el sistema lanzaría una excepción *NetworkOnMainThreadException*. Al utilizar el método *.enqueue()*, delegamos la tarea a un hilo secundario, garantizando que la app siga respondiendo mientras espera los datos de Geoapify.

```

private void consultarGeoapify(double lat, double lon) {
    if (lat == 0 || lon == 0) return;

    OkHttpClient client = new OkHttpClient();

    // 1. Construimos los parámetros por separado para evitar errores de formato
    String coordenadas = lon + "," + lat; // Geoapify exige Longitud, Latitud
    String radio = "2000";

    // 2. Construimos la URL usando concatenación limpia
    // IMPORTANTE: Asegurar de que no haya espacios en blanco en la API_KEY
    String url = "https://api.geoapify.com/v2/places?categories=service.financial.atm" +
        "&filter=circle:" + lon + "," + lat + ",2000" +
        "&bias=proximity:" + lon + "," + lat +
        "&limit=10" +
        "&apiKey=" + API_KEY.trim();

    Log.d("tag: \"PECUNIA_URL\"", "msg: \"URL Final: \" + url);

    Request request = new Request.Builder()
        .url(url)
        .build();

    client.newCall(request).enqueue(new Callback() {
        @Override
        public void onFailure(@NonNull Call call, @NonNull IOException e) {
            runOnUiThread(() -> tvLista.setText("Fallo de red: " + e.getMessage()));
        }
    });
}

```

6.3.3 Parseo de JSON y Actualización de la interfaz

La respuesta del servidor llega en un formato de texto plano llamado JSON, que la aplicación debe interpretar.

La respuesta se procesa mediante los objetos `JSONObject` y `JSONArray`. Se realiza un recorrido por la estructura de datos para extraer los campos *name* (nombre del banco) y *distance* (distancia del usuario).

Un detalle técnico fundamental es el uso de `runOnUiThread`: debido a que la respuesta llega de un hilo secundario de red, Android prohíbe actualizar elementos visuales desde allí. Este método permite “saltar” de vuelta al hilo de la interfaz para mostrar resultados al usuario.

```

@Override
public void onResponse(@NonNull Call call, @NonNull Response response) throws IOException {
    // Si sale Error 400, vamos a leer qué dice el cuerpo del error
    if (!response.isSuccessful()) {
        String errorBody = response.body() != null ? response.body().string() : "Sin detalle";
        Log.e( tag: "PECUNIA_ERROR", msg: "Cuerpo del error: " + errorBody);
        runOnUiThread() -> tvLista.setText("Error 400: Revisa la consola (Logcat) para el detalle.");
        return;
    }

    try {
        String res = response.body().string();
        JSONObject json = new JSONObject(res);
        JSONArray features = json.getJSONArray( name: "features");

        StringBuilder sb = new StringBuilder();
        if (features.length() == 0) {
            sb.append("No se han encontrado cajeros en 2km.");
        } else {
            for (int i = 0; i < features.length(); i++) {
                JSONObject prop = features.getJSONObject(i).getJSONObject( name: "properties");
                String banco = prop.optString( name: "name", fallback: "Cajero");
                String dist = prop.optString( name: "distance", fallback: "?"); // distancia en metros

                sb.append(" 🏧 ").append(banco.toUpperCase()).append("\n");
                sb.append(" 📍 Aprox. a ").append(dist).append(" metros\n");
                sb.append("-----\n\n");
            }
        }

        String finalResult = sb.toString();
        runOnUiThread() -> tvLista.setText(finalResult);

    } catch (Exception e) {
        runOnUiThread() -> tvLista.setText("Error al leer datos.");
    }
}

});
}

```

6.4 Sistema de Alertas y Notificaciones Push(Firebase Cloud Messaging)

Para garantizar que el usuario reciba información relevante (como alertas bajo saldo o recordatorios) incluso cuando la aplicación está cerrada, se ha implementado un servicio de mensajería en la nube basado en FCM(Firebase Cloud Messaging).

6.4.1 Implementación del Servicio en Segundo Plano

A diferencia de las actividades normales, esta lógica reside en una clase que extiende de `FirebaseMessagingService`³. Esto permite que el sistema operativo despierte a la aplicación únicamente cuando llega un mensaje desde el servidor.

```
public void onMessageReceived(RemoteMessage remoteMessage) {  
    if (remoteMessage.getNotification() != null) {  
        mostrarNotificacion(  
            remoteMessage.getNotification().getTitle(),  
            remoteMessage.getNotification().getBody()  
        );  
    }  
}
```

Este método `onMessageReceived` actúa como un “oyente” constante. Cuando la nube envía un paquete de datos, el sistema extrae el título y el cuerpo del mensaje mediante `remoteMessage.getNotification().getTitle()`. Es una operación totalmente transparente para el usuario hasta que se invoca la interfaz visual de la notificación.

6.4.2 Construcción de la Notificación y Canales de Alerta

Desde Android 8.0(API 26), la gestión de notificaciones es más estricta por motivos de privacidad y control del usuario.

En este fragmento que vemos a continuación se destacan tres hitos de programación:

³ En el ecosistema Android, un Service es un componente que permite ejecutar operaciones de larga duración en segundo plano, sin necesidad de que el usuario tenga una interfaz visible abierta. Es ideal para procesos como la escucha de mensajes en la nube.

```
private void mostrarNotificacion(String title, String body) {
    String channelId = "pecunia_canal_alertas";
    Intent intent = new Intent( packageContext: this, Entrar.class);
    intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);

    PendingIntent pendingIntent = PendingIntent.getActivity( context: this, requestCode: 0, intent,
        flags: PendingIntent.FLAG_ONE_SHOT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder( context: this, channelId)
        .setSmallIcon(R.drawable.fondocerditoverde) // Icono de la app
        .setContentTitle(title)
        .setContentText(body)
        .setAutoCancel(true)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent);

    NotificationManager manager = (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel channel = new NotificationChannel(channelId, name: "Alertas Pecunia", NotificationManager.IMPORTANCE_DEFAULT);
        manager.createNotificationChannel(channel);
    }

    manager.notify( id: 101, builder.build());
}
```

1. Uso de PendingIntent⁴: No se lanza un Intent directo, sino un “intento pendiente”. Esto concede permiso al sistema Android para que, cuando el usuario pulse la notificación, abra la actividad de Entrar(Login) con los privilegios de la aplicación Pecunia.
2. Configuración del NotificationChannel: Se define el channelId como “pecunia_canal_alertas”. Es obligatorio registrar este canal en el *NotificationManager* para que el sistema operativo acepte mostrar la notificación.
3. Compatibilidad de Versiones: Se incluye una estructura condicional if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) para asegurar que la app funcione correctamente tanto en dispositivos antiguos como en los más modernos, siguiendo las mejores prácticas de desarrollo.

⁴ Es un contenedor para un Intent que permite a una aplicación externa(en este caso, el Sistema Operativo Android) ejecutar el código de nuestra app como si fuera nosotros mismos.

7. Plan de Pruebas

En este capítulo se describen las pruebas funcionales realizadas sobre la aplicación Pecunia para asegurar su estabilidad, seguridad y correcto funcionamiento de la lógica de negocio.

7.1 Pruebas de Sistema y Gestión de Usuarios

En este apartado se detalla el proceso de verificación del sistema de seguridad de Pecunia. El objetivo es asegurar que la gestión de identidades sea infalible, evitando tanto el acceso no autorizado como la corrupción de la base de datos con información incompleta.

7.1.1 Protocolo de Pruebas Ejecutadas

Se han diseñado cuatro casos críticos que cubren el ciclo de vida inicial del usuario:

ID	ESCENARIO	ENTRADA DATOS	COMPORTAMIENTO ESPERADO	RESULTADO
P1.1	Alta de nuevo perfil	Email válido, Password robusto y selección de Plan	Creación exitosa en Firebase y salto al Dashboard	APTO
P1.2	Intento de Registro inseguro	Password de 4 caracteres	El sistema debe detectar la vulnerabilidad y frenar el proceso	APTO
P1.3	Login de usuario existente	Credenciales registradas previamente	Recuperación del UID y acceso a los datos financieros	APTO
P1.4	Login fallido	Email correcto pero contraseña errónea	Bloqueo de acceso mensaje de advertencia al usuario	APTO

Identificador	Proveedores	Fecha de creación ↓	Fecha de acceso	UID del usuario
pablo@ejemplo.com		12 may 2026	12 may 2026	sx493tfrRkRC7FRYfwOhxwed...

Figura 17. Consola de Firebase Auth con el nuevo usuario creado

Como se evidencia en las capturas, el proceso de registro garantiza la sincronización inmediata entre el cliente(Android) y el backend(Firebase), asignando el UID necesario para operaciones futuras.

→ Validación de Reglas de Seguridad

En el caso P1.2, se puso a prueba el filtro de seguridad local antes de la petición al servidor.

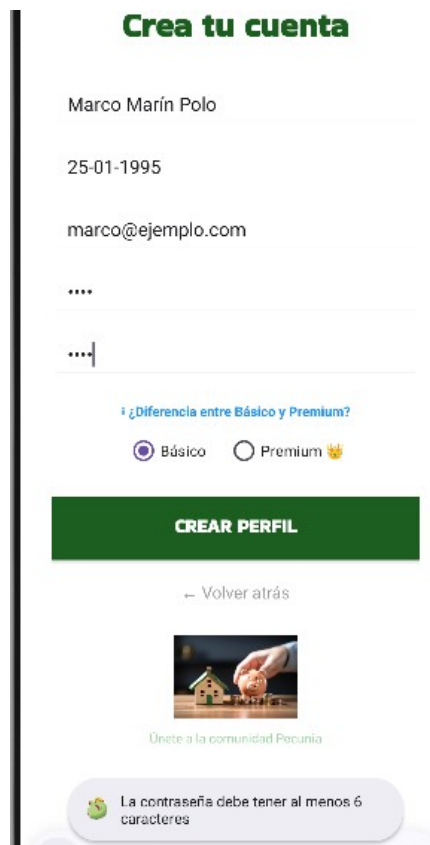


Figura 18. Contraseña corta



Figura 19. Confirmación de contraseña en formulario fallida

→ Flujo de Acceso Exitoso (Login)

El caso P1.3 confirma que el “camino feliz” del usuario es fluido.

Identifícate





Figura 21. Login correcto y menú principal

Figura 20. Usuario con credenciales correctas

La transcripción inmediata entre la actividad de Login y el Dashboard confirma que la sesión se gestiona correctamente y que el UID es recuperado con éxito para filtrar los datos del usuario.

→ Gestión de acceso no autorizado

Para validar el caso P1.4, se realizó un intento de acceso utilizando un correo electrónico registrado pero introduciendo una contraseña incorrecta de forma deliberada.

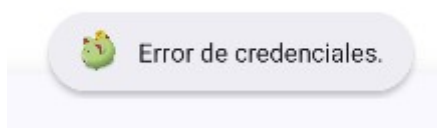


Figura 22. Error al entrar en la app

Como se observa en la evidencia, el sistema no permite acceso a la actividad principal si la respuesta de Firebase Auth es negativa. Se gestiona la excepción mediante un bloque *onCompleteListener*, manteniendo al usuario en la pantalla de entrada para que pueda corregir sus datos, protegiendo así la privacidad de la cuenta.

7.2 Pruebas de lógica de Negocio y control Premium

Aquí se demuestra que el campo tipo de la base de datos realmente controla la aplicación.

7.2.1 Protocolo de pruebas realizadas

ID	ESCENARIO	ENTRADA	COMPORTAMIENTO ESPERADO	RESULTADO
P2.1	Acceso Restringido(Básico)	Pulsar en categoría "Ocio"	Bloqueo de navegación y aviso Toast	APTO
P2.2	Acceso autorizado(Premium)	Pulsar en categoría "Ocio"	Apertura del formulario de gasto personalizado	APTO
P2.3	Personalización interfaz	Login con cuenta premium	Título y algunos elementos con corona y color oro	APTO

7.2.2 Análisis de Evidencias Visuales

→ Accesos restringidos y permitidos según el Usuario

Se demuestra que la aplicación responde correctamente al tipo de Usuario según lo elija en el formulario de registro y capte en la base de datos el tipo.



Figura 23. Utilidades Premium

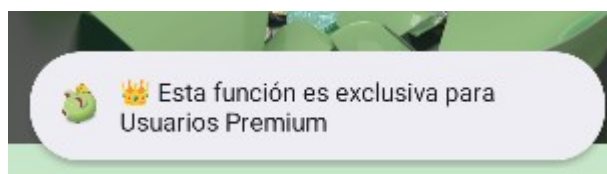


Figura 24. Intento de uso por parte de Usuario Básico a función Premium

→ Diseño exclusivo de Interfaz Gráfica

Según el usuario que ingrese con sus credenciales al menú y las diferentes funcionalidades que tiene la app, la interfaz se va a mostrar de manera diferente para cada uno.



Figura 25. Menú gastos básico



Figura 26. Menú gastos Premium



Figura 27. Bienvenida Básico

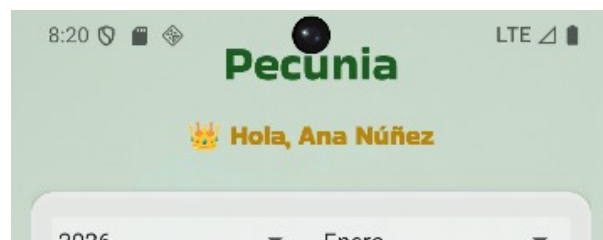


Figura 28. Bienvenida Premium

7.3 Pruebas de integración con Servicios Externos

7.3.1 Protocolos de pruebas realizadas

La conexión con el mundo exterior (Geoapify) funciona.

ID	ESCENARIO	ENTRADA	COMPORTAMIENTO ESPERADO	RESULTADO
P3.1	Búsqueda sin GPS	Abrir sección Cajeros	Mensaje solicitando Ubicación	APTO
P3.2	Consumo de webService	Abrir sección Cajeros	Listado real de bancos cercanos	APTO

7.3.2 Análisis de Evidencias Visuales

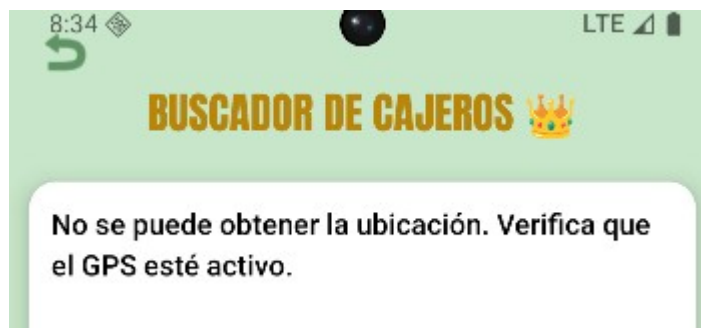
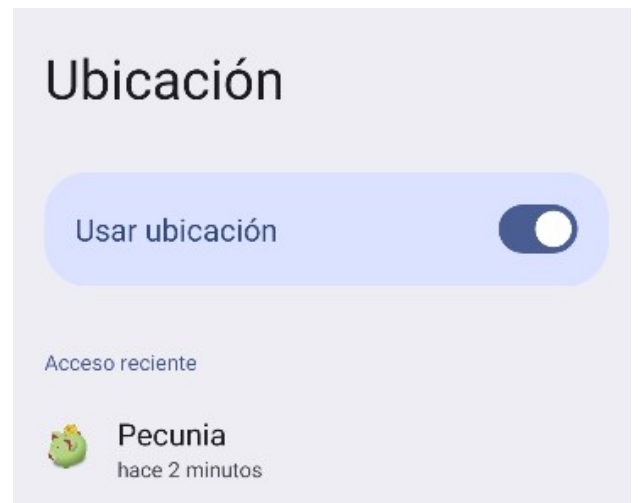


Figura 29. Servicio sin Ubicación activada



Figuras 30 y 31. Ubicación activada con permiso a la aplicación y listado de cajeros

8. Conclusiones

Llegar al final de este proyecto supone mucho más que haber terminado una aplicación funcional; ha sido un proceso de maduración técnica y personal. Si echo la vista atrás, uno de los aspectos que más he disfrutado ha sido el proceso de diseño y la creación de la interfaz. Aprender a manejar los diferentes componentes de AndroidStudio para que la app no solo sea útil, sino también visualmente atractiva, ha sido una experiencia muy gratificante. Ver cómo un esquema en papel o cada idea en la mente se haga realidad en una pantalla interactiva, usando diferentes métodos y herramientas visuales. Todo esto me ha dado una visión mucho más amplia de lo que significa desarrollo de software.

Desde el punto de vista técnico, la curva de aprendizaje con Firebase ha sido uno de los pilares de este trabajo. Descubrir el potencial de persistencia de datos en tiempo real y la gestión de autenticación me ha permitido entender cómo funcionan algunas de las aplicaciones profesionales hoy día. No obstante, el camino no ha estado exento de dificultades. Durante el desarrollo he tenido que enfrentar errores y fallos en el funcionamiento, especialmente al gestionar la asincronía y las peticiones a la API Geoapify. Hubo momentos de frustración cuando el código no respondía como esperaba o cuando los hilos de ejecución chocaban al intentar actualizar la interfaz, pero solventar esos errores mediante el uso de callbacks y un estudio del Logcat ha sido donde más he aprendido.

Este proyecto me ha enseñado que la programación no es solo escribir líneas de código, sino saber conectar diferentes piezas: desde la lógica de negocio hasta la integración de servicios externos y notificaciones en segundo plano. He aprendido a ser más paciente y analítica, entendiendo que cada fallo es una oportunidad para robustecer el sistema.

De cara al futuro soy consciente que Pecunia tiene gran potencial de crecimiento. Me gustaría seguir evolucionando la herramienta integrando más funciones o librerías de gráficos para que el usuario pueda ver su progreso financiero. Queda abierta una puerta para integrar un amplio abanico de ideas y oportunidades para hacer de esta aplicación un sistema más profesional enfocado a las necesidades que le vayan surgiendo a los Usuarios. En definitiva, este proyecto no es un punto y final, sino la base sólida para seguir creciendo como desarrollador, habiendo disfrutado de cada fase de aprendizaje.

9. Anexos

El código fuente íntegro del proyecto, incluyendo archivo de documentación y manual de usuario, se encuentra en mi plataforma de GitHub.

El uso de esta herramienta ha sido también fundamental para el control de versiones y el mantenimiento de la integridad del software ante cambios estructurales.

- Enlace al repositorio: <https://github.com/anuflo/Pecunia-App.git>
- Nombre del Proyecto: Pecunia – Financial Management App
- Rama principal: master/main.

9.1 Glosario de términos

- **IDE (Android Studio):** Entorno de desarrollo integrado utilizado para la programación.
- **Firebase:** Plataforma en la nube de Google para backend y bases de datos.
- **NoSQL (Firestore):** Base de datos no relacional basada en documentos.
- **API (Geoapify):** Interfaz de programación que permite obtener datos externos de geolocalización.
- **JSON:** Formato ligero de intercambio de datos.

9.2 Bibliografía y Recursos

- Documentación oficial de Android Developers.
- Documentación de Firebase para Android.
- Guía de estilo de Material Design.



PECUNIA - Financial Management App